1.12 SSH Keys, known_hosts and authorized_keys

Topic 12 of 13 | ~4 hours | Difficulty: Intermediate

1 What it is

SSH (Secure Shell) is the encrypted protocol, on TCP port 22 by default, used to log in to and run commands on remote machines, and to copy files with `scp`, `sftp` and `rsync`. It replaces older unencrypted tools such as telnet. SSH solves two separate trust problems, and each has its own file:

1. **Does the server prove it is the server I meant to reach?** (host verification). The first time you connect, SSH shows the server's **host key fingerprint**; you accept it, and the client remembers it in `~/.ssh/known_hosts`. Next time, a different key triggers a loud warning — it may be a man-in-the-middle attack.
2. **Does the client prove it is an allowed user?** (user authentication). With **public-key authentication** you generate a **keypair**: a *private key* that never leaves your machine and a *public key* you copy to the server's `~/.ssh/authorized_keys`. The server sends a challenge only the private key can answer, so no password crosses the network and there is nothing to guess or brute-force.

2 Why it matters in DevOps

- **SSH keys are how humans and automation get into servers.** CI/CD deploy steps, Ansible, `rsync` backups and Git over SSH (`git@github.com`) all use them.
- **A leaked or unmanaged private key is a standing breach.** Keys without passphrases on laptops, shared "team keys", and keys that nobody removed after someone left are among the most common real-world access problems.
- **Brute-force noise is constant.** Any public server on port 22 receives automated password-guessing within minutes of coming online; key-only login removes that whole risk class.
- **Rebuilt servers change host keys.** Autoscaling and reprovisioning make "WARNING: REMOTE HOST IDENTIFICATION HAS CHANGED" a daily event; you must know when it is harmless and when it is an attack.

- **Permissions are enforced.** sshd silently refuses keys if `~/.ssh` or `authorized_keys` have the wrong owner or modes (File Permissions §1.6) — the most common reason "my key doesn't work".

3 Real-world system example

GitHub and AWS EC2. GitHub lets you add public SSH keys to your account (Settings, SSH and GPG keys) and then serves Git over `git@github.com`: you authenticate with the private key and GitHub maps the matching public key to your account. GitHub publishes its SSH host key fingerprints in its documentation precisely so that you can compare them with what `ssh` shows on first connect.

On **AWS EC2**, when you launch an instance you name a *key pair*; AWS places the public half in `~/.ssh/authorized_keys` of the image's default user (`ubuntu` on Ubuntu AMIs, `ec2-user` on Amazon Linux), and password login is disabled. You keep the private `.pem` file. Newer practice goes further: AWS Systems Manager Session Manager and EC2 Instance Connect avoid long-lived keys altogether, and large organizations use SSH certificates signed by a certificate authority with short lifetimes. The mechanics you learn here are the base for all of those.

4 Diagram

First-connect host verification and key authentication, step by step:

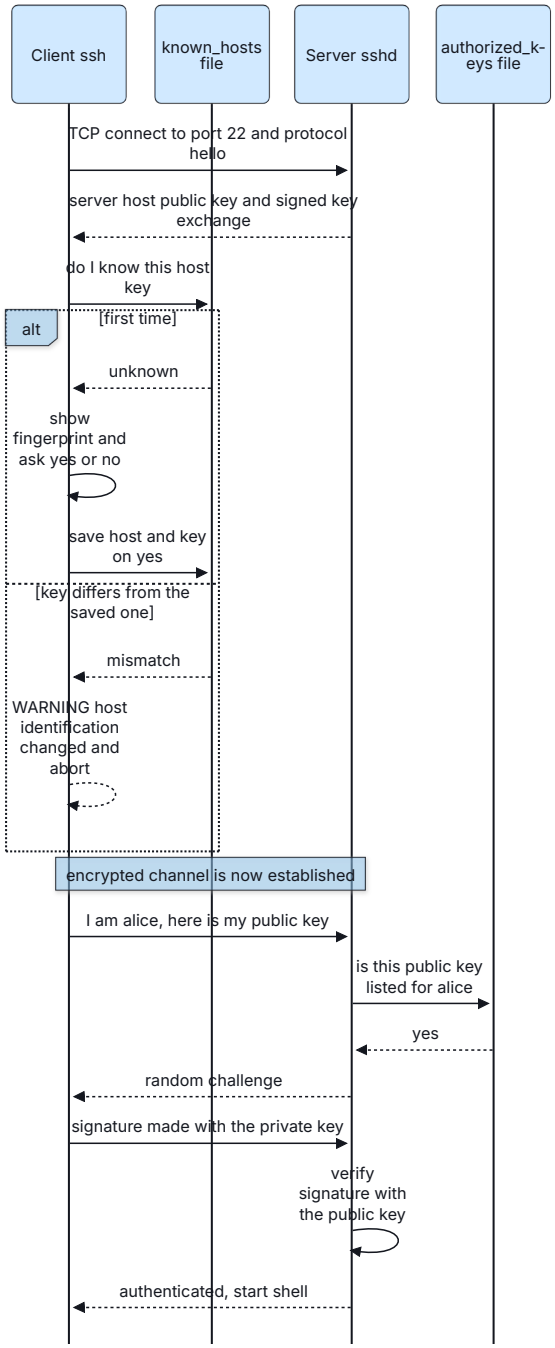

Client ssh
known_hosts file
Server sshd
authorized_keys file
TCP connect to port 22 and protocol hello
server host public key and signed key exchange
do I know this host key
alt
[first time]
unknown
show fingerprint and ask yes or no
save host and key on yes
[key differs from the saved one]
mismatch
WARNING host identification changed and abort
encrypted channel is now established
I am alice, here is my public key
is this public key listed for alice
yes
random challenge
signature made with the private key
verify signature with the public key
authenticated, start shell

Figure 1.12-1

Who holds what:

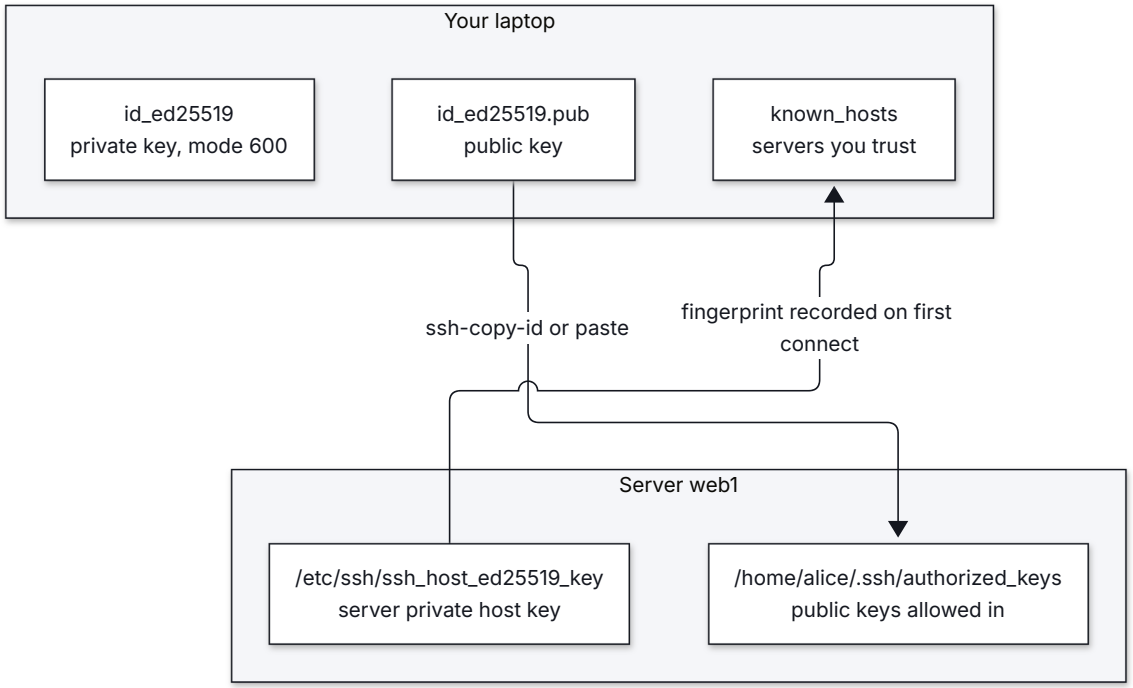


Figure 1.12-2

5 How it works

Key types

Type	Command	Notes
Ed25519	`ssh-keygen -t ed25519`	Modern default: short keys, fast, strong. Use it unless a system cannot
RSA	`ssh-keygen -t rsa -b 4096`	Universally supported; use at least 3072 bits (4096 is common)
ECDSA	`-t ecdsa`	Works, but Ed25519 is generally preferred
DSA	—	Obsolete; disabled by default in current OpenSSH

Files you must know

File	Where	Purpose	Mode
`~/.ssh/id_ed25519`	client	**private** key — never share, never commit	`600`
`~/.ssh/id_ed25519.pub`	client	public key — safe to share, paste into GitHub/servers	`644`

File	Where	Purpose	Mode
`~/.ssh/known_hosts`	client	host keys of servers you've verified	`644`
`~/.ssh/config`	client	per-host shortcuts (see the next topic)	`600`
`~/.ssh/authorized_keys`	server, in the **target user's** home	public keys allowed to log in as that user	`600`
`~/.ssh/`	both	the directory itself	`700`
`/etc/ssh/sshd_config`	server	daemon configuration	`644`, root-owned
`/etc/ssh/ssh_host_*_key(.pub)`	server	the server's own identity	private `600`

The server also requires that the **home directory is not group- or world-writable** (`StrictModes yes`). If any of these are wrong sshd logs `Authentication refused: bad ownership or modes for file …` and the login falls through to the next method.

Anatomy of an `authorized_keys` line

```
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIHxPNGxaIH6gpNrLCTFwVjYvndcJ4+cx/b1YbyBxMvcO
  alice@laptop
^type       ^base64 public key
  ^comment (free text)
```

One key per line. Optional **options** go before the key type and can restrict what the key may do:

```
command="/usr/local/bin/backup.sh",no-pty,no-port-forwarding,no-agent-
  forwarding,from="203.0.113.10" ssh-ed25519 AAAA... ci-backup
```

Option	Effect
`command="..."`	the key can only run this command, whatever the client asks
`from="203.0.113.10,*.example.com"`	only accepted from those addresses
`no-pty`, `no-port-forwarding`, `no-agent-forwarding`, `no-X11-forwarding`	switch off capabilities
`restrict`	switch off all of the above at once; then re-allow only what you need

Host keys, `known_hosts` and fingerprints

- A fingerprint is a hash of a public key, e.g. `SHA256:D8RtMb…`. Compare it with an **out-of-band** source (cloud console output, the provider's documentation, an admin) before typing `yes`.
- `known_hosts` entries may be hashed (`HashKnownHosts yes`, the Debian/Ubuntu client default) so the file does not reveal your server list.
- When a server is legitimately rebuilt, remove the stale entry: `ssh-keygen -R web1.example.com`, reconnect, and verify the new fingerprint.
- `StrictHostKeyChecking`: `ask` (default, prompts), `accept-new` (trusts unknown hosts but still blocks changed keys — a good compromise for automation), `yes` (fail unless already known), `no` (**never use**: disables the protection).

Passphrases

A passphrase encrypts the private key file on disk. If the laptop is stolen or the file leaks, the key is useless without it. Typing it every time is avoided with `ssh-agent`, covered in ssh-agent and Port Forwarding §1.13. Automation keys that cannot use a passphrase should be single-purpose, restricted with the options above, and stored in a secrets manager.

6 Step-by-step guide

Prerequisites: an SSH client (macOS, Linux, WSL2 and recent Windows have it) and a server you can reach by password or console once: a cheap VPS, a cloud VM, a local VM, or a local container with `openssh-server`. Nothing here costs money on a local VM.

1. **Check what you have.**

   ```bash
   ssh -V
   ls -la ~/.ssh
   ```

   `ssh -V` prints something like `OpenSSH_9.6p1 Ubuntu…`. If `~/.ssh` is missing, step 2 creates it.

2. **Generate an Ed25519 keypair with a passphrase.**

   ```bash
   ssh-keygen -t ed25519 -C "alice@laptop-2026" -f ~/.ssh/id_ed25519
   ```

   Enter a passphrase when asked. Expected: it prints `Your public key has been saved in …/id_ed25519.pub` and a randomart image. The `-C` comment is only a label.

3. **Inspect both halves.**

```bash
ls -l ~/.ssh/id_ed25519 ~/.ssh/id_ed25519.pub   # private is -rw------- (600), public
  -rw-r--r--
cat ~/.ssh/id_ed25519.pub                        # one line: ssh-ed25519 AAAA...
  alice@laptop-2026
ssh-keygen -lf ~/.ssh/id_ed25519.pub             # SHA256 fingerprint
head -1 ~/.ssh/id_ed25519                        # -----BEGIN OPENSSH PRIVATE KEY-----
```

   Only the `.pub` file is ever copied anywhere.

4. **Install the public key on the server.** The easy way (uses your existing password login once):

```bash
ssh-copy-id -i ~/.ssh/id_ed25519.pub alice@192.0.2.10
```

   Manual equivalent, useful when you only have console access:

```bash
# run ON THE SERVER as alice
install -d -m 700 ~/.ssh
echo 'ssh-ed25519 AAAAC3...yourpublickey... alice@laptop-2026' >>
  ~/.ssh/authorized_keys
chmod 600 ~/.ssh/authorized_keys
```

   (`192.0.2.10` is a documentation-only address; use your server's real IP or name.)

5. **Verify the host key on first connect.** On the server console (or from the cloud provider's log) print the fingerprint, then connect and compare:

```bash
# on the server
ssh-keygen -lf /etc/ssh/ssh_host_ed25519_key.pub
# on your laptop
ssh alice@192.0.2.10
```

   You will see `The authenticity of host '192.0.2.10 (192.0.2.10)' can't be established. ED25519 key fingerprint is SHA256:… Are you sure you want to continue connecting (yes/no/[fingerprint])?` The fingerprints must match before you answer `yes`. Afterwards `ssh-keygen -F 192.0.2.10` shows the saved entry.

6. **Log in with the key and confirm which method was used.**

```bash
ssh -v alice@192.0.2.10 2>&1 | grep -E "Offering public key|Server accepts
  key|Authenticated to"
```

   Expected: `Offering public key: …id_ed25519`, `Server accepts key`, `Authenticated to … using "publickey"`.

7. **Provoke and read a host-key warning.** Simulate a rebuilt server by editing the saved entry, or rebuild a test VM. When you see `WARNING: REMOTE HOST IDENTIFICATION HAS CHANGED!`, read the message: it names the file and line and prints the remove command. After you have verified out-of-band that the rebuild is expected:

```bash
ssh-keygen -R 192.0.2.10
ssh alice@192.0.2.10        # accept the new fingerprint after verifying it
```

8. **Break and fix permissions on the server.**

```bash
# on the server, as alice
chmod 666 ~/.ssh/authorized_keys
# from the laptop: ssh alice@192.0.2.10   → falls back to a password prompt or
  "Permission denied (publickey)"
sudo journalctl -u ssh -n 5 --no-pager     # or: sudo tail /var/log/auth.log
# look for: Authentication refused: bad ownership or modes for file
  /home/alice/.ssh/authorized_keys
chmod 600 ~/.ssh/authorized_keys           # fix
```

   (Service name: `ssh` on Debian/Ubuntu, `sshd` on RHEL-family.)

9. **Restrict a key to one command.** On the server, prefix the line in `authorized_keys`:

```
command="/usr/bin/uptime",no-pty,no-port-forwarding,no-agent-forwarding ssh-ed25519
  AAAA... alice@laptop-2026
```

   From the laptop, `ssh alice@192.0.2.10 whoami` now prints `uptime` output instead — the server ignored the requested command. Remove the prefix afterward.

10. **Turn off password login once keys work** (keep a second session open!). In

`/etc/ssh/sshd_config.d/10-hardening.conf`:

```
PasswordAuthentication no
PermitRootLogin no
PubkeyAuthentication yes
```

```bash
sudo sshd -t && sudo systemctl reload ssh      # validate config first, then reload
```

From a **new** terminal confirm you can still log in before closing the old session. Never reload sshd without testing from a second connection.

11. **Rotate the passphrase and remove an old key.**

```bash
ssh-keygen -p -f ~/.ssh/id_ed25519               # change passphrase
# on the server, remove one line by comment (check with grep first)
grep -n 'alice@old-laptop' ~/.ssh/authorized_keys
sed -i '/alice@old-laptop/d' ~/.ssh/authorized_keys
```

7 Real-world scenario

(Illustrative.) A contractor leaves the project on Friday. On Monday a security review asks: "Who can still log in to production as `deploy`?" Nobody knows — the `authorized_keys` file on four servers has eleven lines, and seven have comments like `key1`, `new` and `laptop`. The team replaces the free-text approach: each key line must carry `name@device-date`; contractor keys carry a `from=` restriction; and the file is generated from a Git repository by configuration management (Configuration Management §0.4) so removing a person is a reviewed pull request that rewrites the file everywhere.

The same week a rebuilt staging host triggers the changed-host-identification warning for a junior engineer, who "fixes" it by setting `StrictHostKeyChecking no` globally. The team reverts that, shows how to compare fingerprints from the cloud console log, and sets `accept-new` for automation only. **Lesson:** SSH access is an inventory problem as much as a crypto one — know whose key each line is, and never silence host-key checks.

8 Common mistakes and best practices

Don't

- Share or email a private key, commit it to Git, or paste it into chat. If it happened, treat the key as compromised and replace it everywhere.

- Create keys without a passphrase on a laptop "to save time".
- Use one shared key for a whole team or for many servers.
- Answer `yes` to a fingerprint without comparing it, or disable host checking with `StrictHostKeyChecking no` / `UserKnownHostsFile /dev/null` outside a throwaway lab.
- Leave `chmod 777` on `~/.ssh` or home directories to "fix" login problems.
- Reload sshd after a config change without validating (`sshd -t`) and testing a second session.

Do

- Use Ed25519 (or RSA 4096), one key per person per device, with an identifying comment.
- Keep private keys at `600`, `~/.ssh` at `700`, and back up with encryption; use a hardware key (FIDO2 `ed25519-sk`) or a password manager's SSH agent for sensitive access.
- Disable password login and root login on servers; allow specific users with `AllowUsers` where practical.
- Audit `authorized_keys` regularly and remove keys of departed people the same day.
- Restrict automation keys with `command=`, `from=` and `restrict`.

9 Tasks

- [ ] **T1 — Make and read a keypair** *(≈20 min, beginner)*
  - **Goal:** understand what each file contains.
  - **Steps:**
    1. Generate `~/.ssh/lab_ed25519` with a passphrase, and an RSA 4096 key `~/.ssh/lab_rsa`.
    2. Show modes with `ls -l`, print both public keys and their fingerprints (`ssh-keygen -lf`).
    3. Derive the public key again from the private one with `ssh-keygen -y -f ~/.ssh/lab_ed25519`.
  - **Done when:** you can point out type, base64 blob and comment in each `.pub` line, and the re-derived public key matches the file.
- [ ] **T2 — Key-based login to a lab server** *(≈30 min, beginner)*
  - **Goal:** log in without a password.
  - **Steps:**
    1. Create a local VM or VPS user `alice` and install your public key with `ssh-copy-id`.

2. Log in with `ssh -i ~/.ssh/lab_ed25519 alice@<host>` and verify with `ssh -v … | grep Authenticated`.

- **Done when:** the output says `Authenticated … using "publickey"` and no account password was asked.

☐ **T3 — Verify a host key properly** *(≈30 min, intermediate)*

- **Goal:** compare fingerprints rather than trusting blindly.
- **Steps:**
  1. On the server run `ssh-keygen -lf /etc/ssh/ssh_host_ed25519_key.pub`.
  2. From the client run `ssh-keyscan -t ed25519 <host> | ssh-keygen -lf -` and compare the two.
  3. Connect with `-o StrictHostKeyChecking=ask` and verify the prompt shows the same fingerprint.
- **Done when:** all three fingerprints are identical and `ssh-keygen -F <host>` returns the saved entry.

☐ **T4 — Break it: changed host key and bad permissions** *(≈40 min, intermediate)*

- **Goal:** diagnose the two most common SSH failures.
- **Steps:**
  1. Regenerate the server's host keys (`sudo rm /etc/ssh/ssh_host_* && sudo dpkg-reconfigure openssh-server`, on a lab VM only) and reconnect. Read the warning, remove the old entry with `ssh-keygen -R`, and reconnect.
  2. On the server run `chmod 666 ~/.ssh/authorized_keys`, try to log in, and find the cause in the sshd log. Fix the mode.
- **Done when:** you fixed both failures and can quote the exact log line for the permissions case.

☐ **T5 — Lock a key down** *(≈40 min, advanced)*

- **Goal:** give automation the least possible power.
- **Steps:**
  1. Create a separate keypair `ci_deploy`. On the server, add it to `authorized_keys` with `command="/usr/local/bin/deploy.sh",restrict,from="<your client IP>"`.
  2. Write a `deploy.sh` that echoes `$SSH_ORIGINAL_COMMAND`. Run `ssh -i ci_deploy alice@host 'cat /etc/shadow'` and observe that only your script runs.
  3. Try to get a shell (`ssh -i ci_deploy alice@host`) and a forwarded port (`-L 8080:localhost:80`).
- **Done when:** the key can run only `deploy.sh`; shell and port forwarding are refused.

- [ ] **T6 — Prove it: harden and hand over** *(≈45 min, advanced)*
  - **Goal:** demonstrate the full lifecycle without notes.
  - **Steps:**
    1. On a fresh lab server, from console access only: create a user, install a key manually with correct modes, disable password and root login, validate with `sshd -t`, reload, and prove it from a second terminal.
    2. Write a 6-line runbook for offboarding: how to find and delete a leaver's key, check for stray keys, and verify they cannot log in.
  - **Done when:** key login works, `ssh -o PubkeyAuthentication=no alice@host` is refused, and a teammate can follow your offboarding runbook.

10 Self-check questions

1. Which key do you copy to the server, and which one must never leave your machine?
2. What two different things do `known_hosts` and `authorized_keys` prove?
3. Your key was installed, but SSH still asks for a password. Name three things to check on the server.
4. A rebuilt server triggers "REMOTE HOST IDENTIFICATION HAS CHANGED". How do you decide whether it is safe?
5. Why is `StrictHostKeyChecking=accept-new` better than `no` for automation?
6. What does a `command=` option in `authorized_keys` do?

ANSWERS

1. The public key (`.pub`) goes to `authorized_keys` ; the private key stays at `600` on your machine.
2. `known_hosts` lets the client verify the server's identity; `authorized_keys` lets the server verify the client's identity.
3. Ownership and modes (`~/.ssh` 700, `authorized_keys` 600, home not group/world-writable); the key is in the right user's home; `PubkeyAuthentication` is enabled; check the sshd log for "bad ownership or modes" and look at `ssh -v` output.
4. Compare the new fingerprint out-of-band (cloud console, provider docs, an admin) with what the client reports. Only after it matches, remove the old entry with `ssh-keygen -R` and reconnect.
5. `accept-new` trusts hosts seen for the first time but still refuses if a known host's key changes; `no` accepts everything and so allows impersonation.
6. It forces that key to run only the given command, ignoring what the client asked for (available as `$SSH_ORIGINAL_COMMAND`).

11 Further reading

- `man ssh`, `man ssh-keygen`, `man sshd`, `man sshd_config`, `man ssh_config` and `man ssh-copy-id` — the authoritative references.
- OpenSSH project documentation (openssh.com), including release notes for deprecated algorithms.
- GitHub Docs, "Connecting to GitHub with SSH" and AWS EC2 documentation on key pairs.
- *SSH, The Secure Shell: The Definitive Guide* (Barrett, Silverman, Byrnes).
- See also: ssh-agent and Port Forwarding §1.13 and Linux and SSH Hardening §11.1.